Lab 4&5 Project Progress Report

Project Title:
Real-time earthquake & aftershock detection using streaming data

Group No: 20
Reporting Period: 31st August - 11th September

1.Project Objective:

The goal is to create an earthquake monitoring and aftershock detection system which would process real-time USGS earthquake data, detect potentially correlated earthquakes via space-time characteristics, and save the data for historical analysis. The architecture of the whole system is based on the stream and lakehouse concepts, and it will contain raw, cleansed, and enriched earthquake data at the levels of Bronze, Silver, and Gold.

The purpose of this sprint, done during August 31 - September 11, was to move from the suggested architecture towards the working data processing pipeline, where we will ingest real-time USGS data, detect earthquakes, stream them via Kafka streams, and process using Spark Structured Streaming and Bronze-Silver-Gold workflow.

---

2. Sprint Plan and scope:

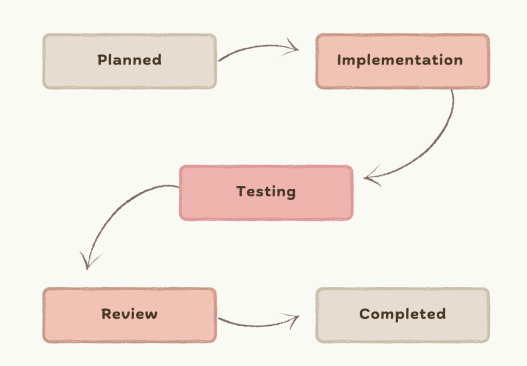
2.1 Sprint Objectives

Objective	Expected Outcome	Priority
Identify real-time data source	Working USGS earthquake data feed	High
Design Streaming Architecture	Kafka based streaming pipeline	High
Implement Data Ingestion	New earthquake events reach kafka	High
Implement stream processing	Structured preprocessing of incoming events	Medium

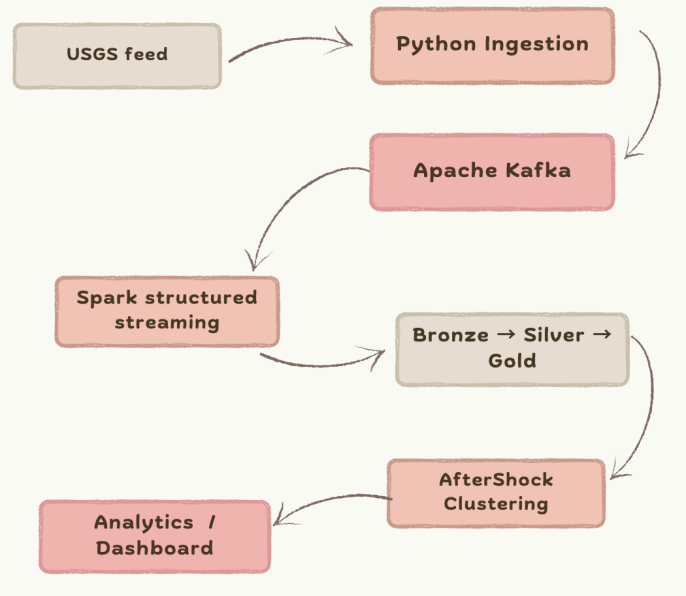
Design after shock detection	Initial spatiotemporal detection methodology	Medium
Implement storage pipeline	Searchable raw and processed earthquake data	Medium

The sprint was all about getting the project from the proposed architecture to becoming a live streaming pipeline. The USGS GeoJSON feed is periodically updated, and our Python ingestion component polls the feed to retrieve earthquake events, but our architecture will utilize Python ingestion, Kafka, Spark Structured Streaming, and Bronze-Silver-Gold Storage.

2.2 Project Workflow



In this sprint, the workflow included the process of discovering and validating the data source → setting up ingestion and streaming → processing and structuring the data → testing the pipeline. The proposed pipeline was:



3.Progress from Aug 31, 2026 to Sep 11, 2026

3.1 Progress Summary

Work Item	Progress	Summary
USGS Data Source	Completed	Selected and tested USGS GeoJSON earthquake feed
Data Retrieval	Completed	Python polling mechanism developed to retrieve new events
Kafka Pipeline	Completed	Kafka topic created for streaming earthquake events
Stream Processing	In Progress	Started development of the Spark Structured Streaming pipeline for processing Kafka events.
Storage	In Progress	Selected MinIO + Delta Lake and started the Docker-based MinIO setup.
Aftershock Detection	Planned	Spatiotemporal clustering approach being designed

3.2 Technical Progress

Data Ingestion

A Python-based ingestion component was developed to retrieve earthquake events from the USGS GeoJSON feed. Event IDs are used to identify newly observed events and avoid repeatedly publishing the same event.

Kafka

The retrieved earthquake events are converted into JSON messages and published to the Kafka `earthquakes` topic.

Storage

For the storage layer, MinIO was selected as the S3-compatible object storage system. MinIO was configured using Docker to provide a reproducible local development environment. Delta Lake was selected as the table-management layer for supporting the planned Bronze, Silver, and Gold data arrangement.

3.3 Evidence

```
import requests

url = "https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_day.geojson"

response = requests.get(url)

data = response.json()

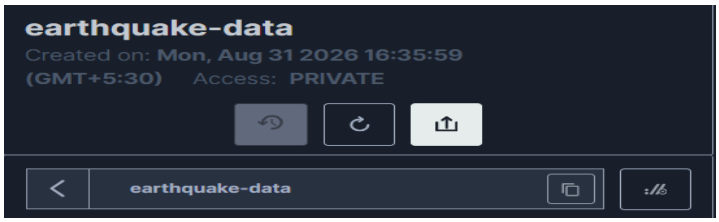
print(data.keys())

dict_keys(['type', 'metadata', 'features', 'bbox'])
```

Successful retrieval of earthquake data from the USGS GeoJSON feed**.**

```
PS C:\Users\shrey\Real-time-Earthquake-Aftershock-Detection-Using-Streaming-Data\storage> docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS
                                                NAMES
6242c201eed8   quay.io/minio/minio:latest     "/usr/bin/docker-ent…"   33 minutes ago   Up 33 minutes   0.0.0.0:9000-9001-
>9000-9001/tcp, [::]:9000-9001->9000-9001/tcp   earthquake-minio
PS C:\Users\shrey\Real-time-Earthquake-Aftershock-Detection-Using-Streaming-Data\storage>
```

MinIO storage service service running through Docker



earthquake-data bucket created in MinIO.

---

4. System Architecture

The data flow of the proposed system begins with USGS live earthquake feed, which offers earthquake data continuously updated in GeoJSON format. This feed includes data like the earthquake ID, timestamp, location, depth, magnitude, and place.

Initially, a Python ingestion service keeps checking the USGS feed periodically and retrieves the relevant earthquake data. It makes use of the earthquake ID to detect any new earthquake and avoid retrieving the same event again and again. The new earthquake events are transformed into JSON objects and posted into the Kafka earthquake topic.

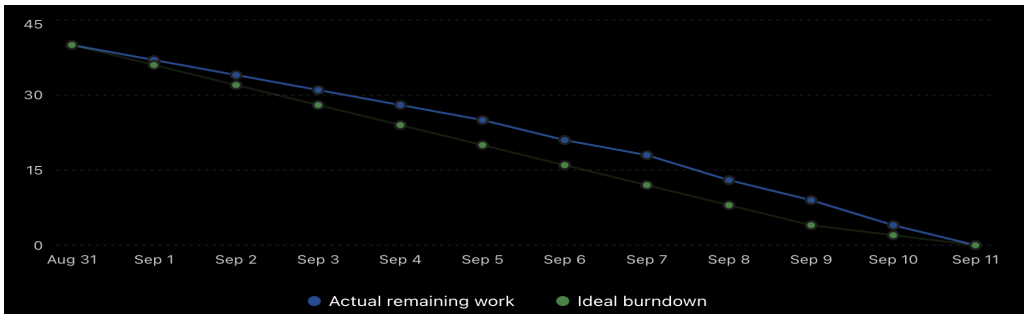
The Kafka Stream is processed by Spark Structured Streaming after being received. The events are stored as they are in the Bronze layer. Then, there is the cleaning and validation phase where duplicate events are filtered out, standardization of the timestamps and the fields is made, and handling of invalid/unexpected data. The cleansed events are stored in the Silver layer.

Then, the data from the Silver layer is used for spatiotemporal clustering. The earthquakes are compared depending on the time, geographical proximity, and the magnitude in order to identify possible mainshock-aftershock relations. This information is gathered and presented as the number of aftershocks, geographical spread, and temporal activity. This enriched data is stored in the Gold layer.

---

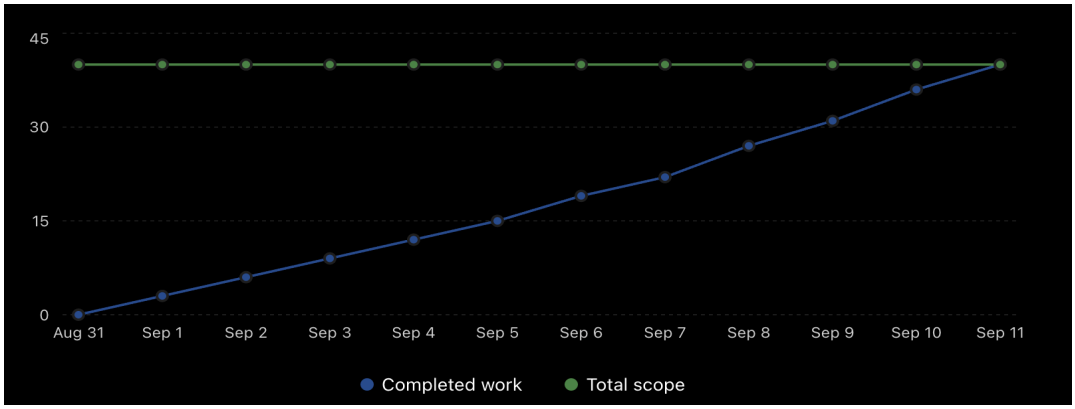
5. Sprint Metrics

5.1 Sprint Burndown



The burndown chart depicts the decrease of work left in the sprint. Actual progress was a bit above the perfect line for the majority of the sprint as some tasks needed more time for implementation and testing. By the end of the sprint, the major planned ingestion and streaming tasks were completed, while Spark processing, storage integration and aftershock detection remained in progress or planned for the next sprint .

5.2 Sprint Burnup

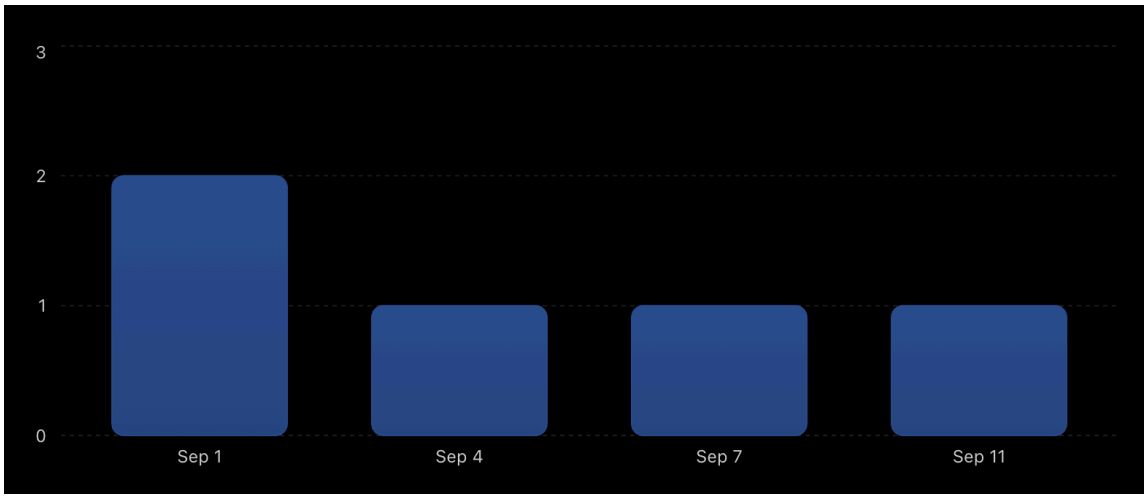


The burnup graph shows the accumulation of completed work throughout the sprint. The completed work increased progressively, while some planned tasks remained in progress or were carried forward to the next sprint. .

5.3 Scope Changes

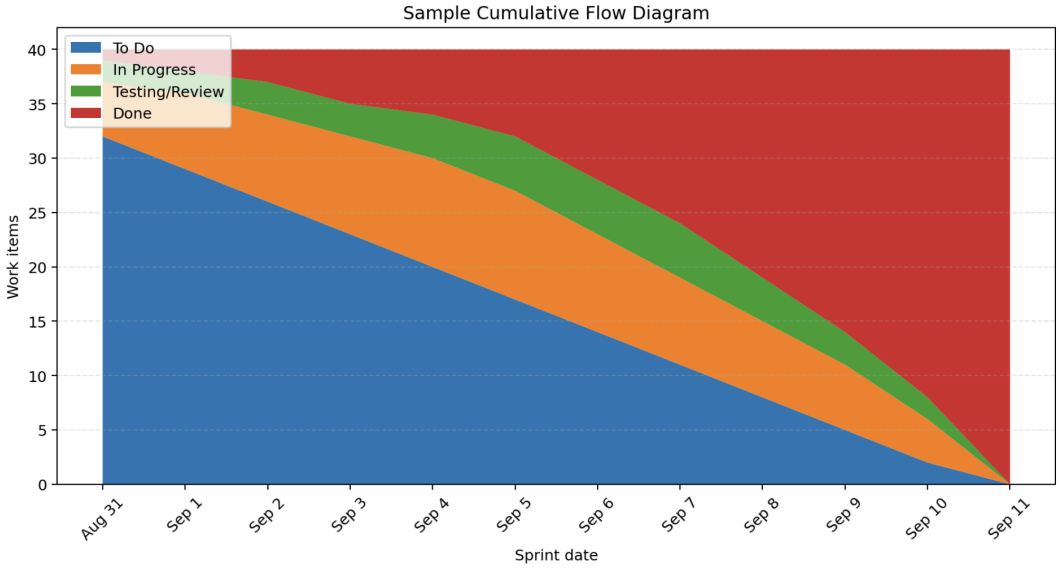
No scope changes have been recorded throughout the sprint. The scope of the sprint was still concerned about the development of the streaming data pipeline involving the data collection from USGS, Kafka streaming, Spark processing, and Bronze-Silver-Gold data architecture.

5.4 Sprint Throughput



It can be seen from the sprint throughput that there have been 5 major work items done during the period when work was completed. Work was done on two tasks on September 1st, then on one task on September 4th, one task on September 7th and another task on September 11th.

5.5 Cumulative Flow Diagram



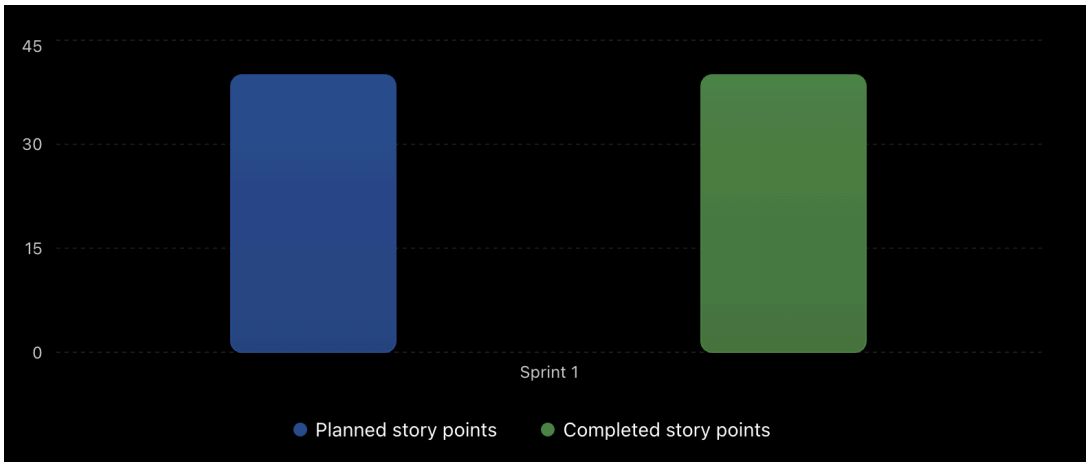
The flowchart reveals the movement of work from “To Do,” to “In Progress,” to “Testing,” to “Done” throughout the sprint. The increase in width of the “In Progress” and “Testing” bands suggests that there is an accumulation of work in these steps, which could suggest a bottleneck problem. As the sprint progresses, there is a marked increase in the “Done” area.

- **Bottleneck:** Widening Testing/Review bands suggest that the testing phase is causing delays in the process.
- **WIP:** In Progress bands refer to work currently in development.
- **Flow:** Moving bands smoothly show how work progresses through the process.
- **Improvement:** Higher Done bands towards the end suggest improved completion and flow.

5.6 Cycle Time

Task	Start	Completion	Time
USGS data ingestion	Sep 1	Sep 1	1 day
Kafka pipeline	Sep 1	Sep 4	4 days
Spark processing	Sep 3	Sep 3	1 day
Data Cleaning	Sep 4	Sep 7	3 days
Storage setup	Sep 9	Sep 11	3 days

5.7 Velocity



The sprint velocity is 40 story points, where the estimated sprint scope is 40 story points while the completion is the same by September 11. The reason why 40 story points can be taken as the basis for the upcoming sprints is because this is the first sprint with regard to story points.

Sprint	**Planned Story Points**	**Completed Story Points**	**Velocity**
Sprint 1 (Aug 31-Sep 11)	40	40	40

6. Result and Key Findings

Metric/Result	Observation	Implication
USGS connectivity	Earthquake feed was successfully retrieved	USGS can act as the project's external data source
Data format	Response was received as structured GeoJSON/JSON	Data can be processed programmatically
Retrieved events	350 events were obtained during testing	The feed provides enough event records for pipeline testing
MinIO deployment	MinIO container successfully ran through Docker	Local object storage is available for the data-lake layer
Storage bucket	earthquake-data bucket was created	The storage layer is ready for integration
Kafka pipeline	Kafka topic was created for streaming earthquake events	Kafka provides the messaging layer for transferring earthquake data to downstream processing

1.Data source from USGS: The USGS GeoJSON feed was successfully connected via Python and structured earthquake data obtained for further processing.

2.Streamability: The obtained earthquakes can be transformed to JSON messages and streamed via the Kafka earthquakes topic.

3.Storage capabilities: MinIO was successfully launched via Docker, and the earthquake-data bucket was created, laying the foundation for the storage layer.

4. Current stage of pipeline: Ingestion and streaming components have been successfully set up, while full Spark processing and aftershocks clustering need to be implemented.

5. Next step: The next step will be dedicated to the integration of the streaming components, aftershock detection, and creating a Bronze-Silver-Gold pipeline.

7. Research and White Papers:

7.1 Research Papers

Reference	Relevant Findings	Application to Project

Paper 1 – Field Stream Data Model for real-time earthquake detection	The system employs Apache Spark and Kafka for processing continuous seismic data using temporal windows. It shows real-time detection of earthquakes and spatial correlation of adjacent earthquakes.	Reinforces the choice of using Kafka and Spark Structured Streaming for the processing of continuous earthquake events. The use of windows is also relevant to our aftershock detection algorithm.
Paper 2 – USGS Earthquake Sequence Product	The USGS categorises earthquakes that are related in time and space to determine earthquake sequences. This facilitates analysis of the relationship between earthquakes and the evolution of aftershock sequences.	Fulfills our goal of finding associated earthquakes based on spatial and temporal properties and provides impetus for developing our aftershock clustering module.

7.2 White Papers

Technical Implementation Plan for the ShakeAlert Production System – An Earthquake Early Warning System for the West Coast of the United States

Authors: Given et al.
Organization: U.S. Geological Survey (USGS)
Year: 2014
Report: USGS Open-File Report 2014–1097

ShakeAlert Technical Report discusses the design of an earthquake warning system created by the USGS and partners. Data is collected using sensors detecting ground motion. Data transfer takes place between sensor and processor, events are detected and alerts are generated for the user. Continuous data gathering, reliable communication, real time processing and event detection have been emphasized as significant in earthquake monitoring systems.

Ground-Motion Sensors

↓

Data Transmission

↓

Central Processing

↓

Event Detection

↓

Alert Generation

↓

End Users

Simplified architecture of the ShakeAlert earthquake

8. Challenges and Design Decisions

Challenge	Impact	Decision/Solution
USGS feed is updated in periods rather than a stream of events	The same earthquake event will be received in multiple poll responses, and the system cannot depend on an event push mechanism	Use Python to do the polling and identify the events using the unique USGS event ID before publishing them on Kafka.
Handling Duplicate Earthquake Events	Duplicate entries may be recorded multiple times and impact subsequent analysis as well as counting of aftershocks.	Use the USGS event id to de-duplicate events so that already processed events are not recorded again.
Scalability considerations for event stream	Sending data directly from Python to the processing layer will tie the two very closely together	Use Apache Kafka as an intermediary messaging layer using the earthquakes topic.
Requirement of Storage of Raw and Processed Data	The traditional approach of using a single storage layer will make it tough to store raw data and do historical analysis.	Use MinIO with Delta Lake with the Bronze-Silver-Gold data approach.

9. Current Status and Next Sprint Plan

Component	Current Status
Data Source	Completed
Kafka / Ingestion	In progress and under testing
Stream Processing	In progress
Aftershock Detection	Planned
Storage / Lakehouse	Partially Implemented
Cluster Statistics & Analytics	Future To Do
Elasticsearch Integration	Future To Do
Dashboard / Visualization	Future To Do
End-to-End Pipeline Testing	Future To Do
Performance & Scalability Testing	Future To Do
Final Deployment	Future To Do

10. Conclusion

The sprint brought consistent progress towards the development of the real-time earthquake and aftershock detection system. Major components of the pipeline were initiated during the sprint, including USGS data ingestion and Kafka-based streaming, while Spark Structured Streaming and the Bronze-Silver-Gold storage workflow are currently under development.

Overall, most of the sprint goals have been reached, and the system is shifting from the concept into the process of implementation. At this point, the system serves as a solid basis for continuous processing of earthquake data; however, additional work needs to be done to finalize the clustering and analytics part of the system.

The following tasks for the next sprint will include: aftershock clustering, completion of Gold-layer analytics, testing of the full pipeline, and dashboard development.

11. References:

1. Whittier, J. C., Nittel, S., & Subasinghe, I. (2017). Real-time earthquake monitoring with spatio-temporal fields. ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences, IV-4/W2, 215–222. **https://doi.org/10.5194/isprs-annals-iv-4-w2-215-2017**
2. Given, D. D., Cochran, E. S., Heaton, T., Hauksson, E., Allen, R., Hellweg, P., Vidale, J., & Bodin, P. (2014). Technical implementation plan for the ShakeAlert production system: An earthquake early warning system for the West Coast of the United States (U.S. Geological Survey Open-File Report 2014–1097). https://doi.org/10.3133/ofr20141097
3. U.S. Geological Survey. (n.d.). Earthquake sequence product. U.S. Department of the Interior. https://www.usgs.gov/programs/earthquake-hazards/science/earthquake-sequence-product

12.Tracker Link:

https://docs.google.com/spreadsheets/d/1t_s9ceWBCumw9T-Ds-D1xpI7wj1Lrhdt4x-Lu2PSHDg/edit?usp=sharing